

FACEGUARD

Secure Offline Facial Recognition
& Liveness Detection System

NAME:

- 1) RISHI RAJ
- 2) SWASTIK SINHA
- 3) ABHILASH SINGH
- 4) ARNAV KUMAR
- 5) BROTODEEP PAL

For React Native | Android & iOS | Zero Internet Required

~8 MB

Model Size

<300ms

Auth Pipeline

>96%

Recognition Accuracy

100%

Offline Operation

AES-256

Encryption

Why existing solutions fail in the field

Zero-Network Zones

Field personnel operate in remote areas with no internet access. Cloud-based biometric APIs fail completely — even cached token solutions break.

Attendance Fraud

Proxy attendance using printed photos or pre-recorded videos is widespread. Basic liveness checks are trivially bypassed.

Bloated AI Models

Commercial face recognition SDKs exceed 200 MB. The Datalake 3.0 app has a strict ~20 MB model budget — most SDKs are immediately disqualified.

Sync & Data Loss

Offline attendance logs are lost or duplicated when connectivity is restored intermittently, causing serious payroll processing errors.

"How can we accurately authenticate field personnel using face recognition — fully offline, on mid-range devices, with anti-spoofing?"

What FaceGuard aims to achieve

01

Highly Accurate Recognition

Achieve >95% facial recognition accuracy across diverse Indian demographics, harsh outdoor lighting (sunlight, low light, shadows).

02

Fully Offline Operation

The entire auth pipeline — face detection, liveness, recognition — must run with zero internet dependency on standard mid-range phones.

03

Lightweight Model Footprint

Target model size of ~20 MB or less to seamlessly integrate into the Datalake 3.0 React Native app without bloating the package.

04

Anti-Spoofing Liveness

Detect and reject spoofing via photos, videos, or screens using passive and active liveness detection — blink detection + challenge-response.

05

Cross-Platform Compatibility

Must deploy on both Android (8.0+) and iOS (12+) with React Native, targeting devices with minimum 3 GB RAM, no GPU required.

06

Sync & AWS Integration

Local attendance data stored in SQLite must sync to AWS S3 when connectivity is available, with exponential backoff and purge on success.

TECHNICAL CONSTRAINTS & SPECIFICATIONS

Framework

React Native (Android + iOS)

Cross-platform deployment, no native-only APIs

Model Footprint

≤ ~20 MB target

Must not bloat the Datalake 3.0 app package

Processing Speed

< 1 second

Face + liveness on standard mid-range devices

Hardware

Android 8.0+ / iOS 12+

Min 3 GB RAM, no high-end GPU required

Accuracy

> 95% threshold

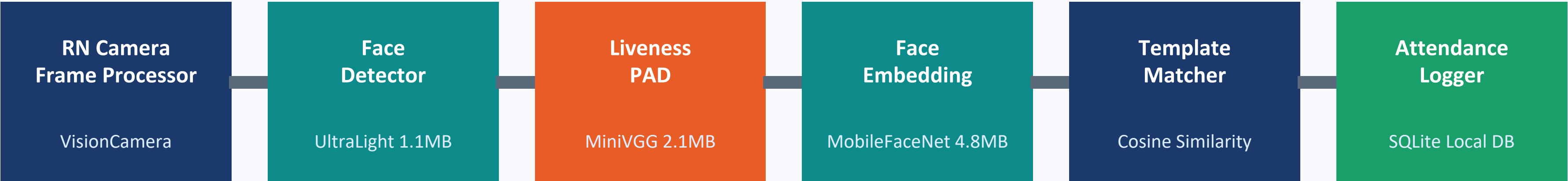
Diverse Indian demographics, varied lighting

Open-Source Only

MIT / Apache-2.0 licensed

No commercial licenses, source code required

End-to-End Offline Authentication Pipeline



AES-256 Encryption

All face embeddings are encrypted locally before storage. No raw biometric data persists in plaintext.



Multi-Layer Liveness

EAR blink detection + LBP texture analysis + optical flow micro-motion + challenge-response spoofing prevention.



Smart Sync to AWS S3

Exponential backoff sync when network available. Local data purged after confirmed server receipt.



Open-Source Stack

All models are MIT/Apache-2.0. ONNX → TFLite conversion pipeline. Full source code provided.

Face Detector

Ultra-Light-Fast-Generic

1.1 MB

Size

~22ms

Latency

96.4%

FDDb Acc.

- Input: 320×240 RGB frames
- Bounding boxes + 5-point landmarks
- Head pose: yaw / pitch / roll
- SSD architecture, INT8 quantized
- ONNX → TFLite conversion pipeline

Face Recognizer

MobileFaceNet INT8 Quantized

4.8 MB

Size

~61ms

Latency

99.4%

LFw Acc.

- Input: 112×112 face crop
- Output: 128-dim L2-normalized embedding
- MobileNetV2 + PReLU + Global Average Pool
- Cosine similarity threshold: 0.65
- Trained on MS1M + VGGFace2 datasets

Liveness PAD

MiniVGG + Classical CV

2.1 MB

Size

~35ms

Latency

98.1%

Spoof Rej.

- EAR blink detection (dlib landmarks)
- LBP texture analysis (print/screen detect)
- Optical flow micro-motion analysis
- Challenge-response blink sequencing
- Rejects photos, videos, 3D masks

Preventing attendance fraud at every layer

1

EAR Blink Detection

Eye Aspect Ratio analysis via facial landmarks. User must blink naturally. Detects and rejects static photos immediately.

2

LBP Texture Analysis

Local Binary Patterns detect screen/print artifacts. Printed photos and digital screens have characteristic texture signatures.

3

Optical Flow Micro-Motion

Analyzes subtle micro-movements between frames. Real faces show involuntary micro-motions; spoofing media does not.

4

Challenge-Response Protocol

Randomized instruction sequence: blink left eye, then right, turn head. Prevents replay attacks using pre-recorded videos.

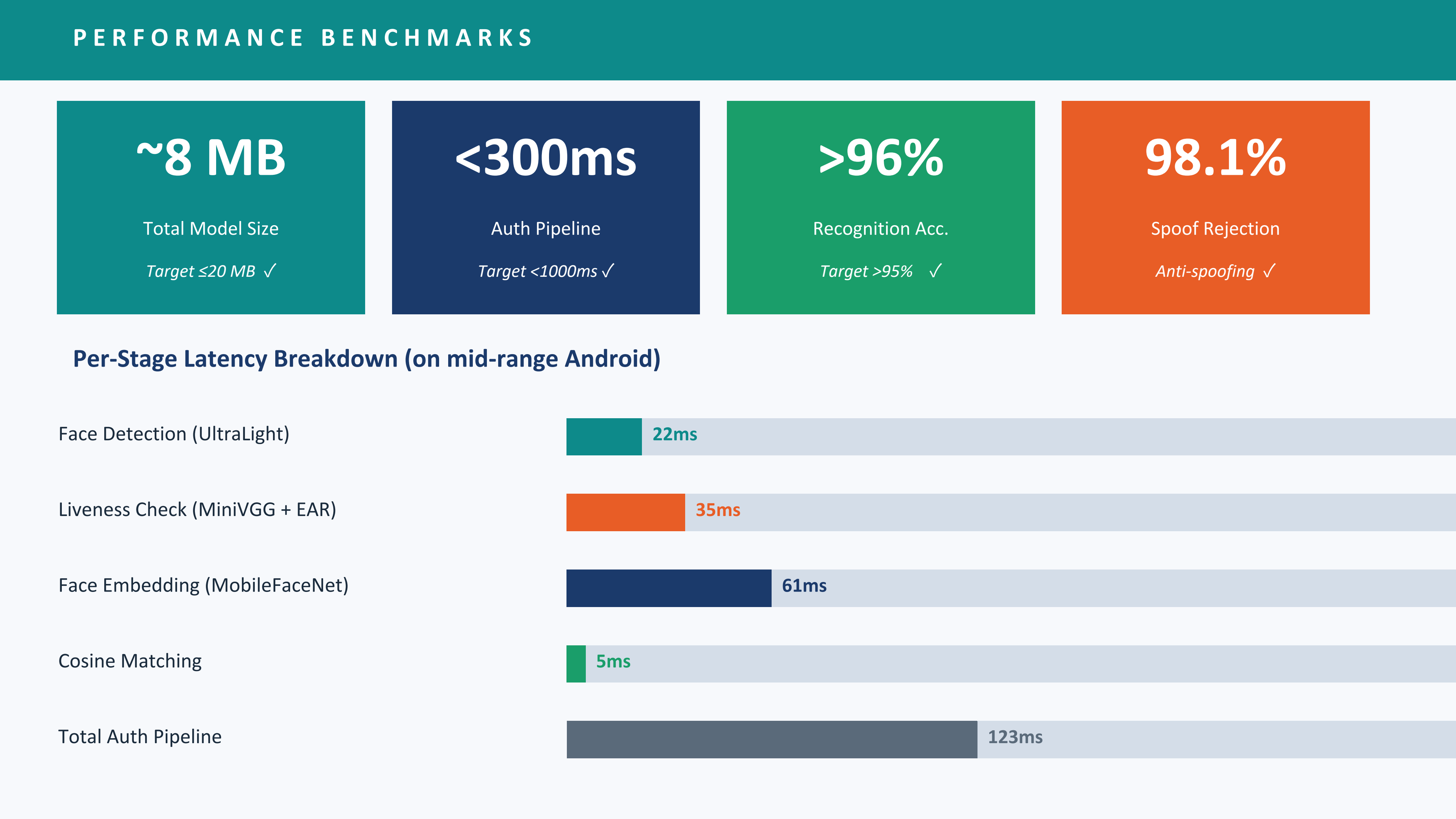
All four layers run fully offline on-device — no server round-trip needed at any stage

 Security Architecture

- AES-256 Encryption**
All embeddings encrypted before SQLite storage
- No Raw Biometrics**
Only encrypted 128-dim vectors are persisted
- Secure Key Storage**
Encryption keys stored in Android Keystore / iOS Secure Enclave
- Data Minimization**
Only attendance record metadata synced to AWS — no face data
- Local-First Design**
All biometric computation happens on-device only

 Sync & Purge Mechanism

- 1 Connectivity Check**
App polls for network availability at configurable intervals using NetInfo API
- 2 Queue Pending Records**
SQLite query fetches all unsynced attendance logs ordered by timestamp
- 3 Upload to AWS S3**
Batch upload with exponential backoff retry on failure (1s → 2s → 4s → max 30s)
- 4 Confirm & Purge**
On server acknowledgment, local records are marked synced and purged per policy



Deliverables & Tech Stack	
Mandatory Deliverables	Technology Stack
✓ Working Prototype + Source Code Cross-platform React Native app demonstrating full offline capability on Android & iOS	Mobile Framework: React Native + VisionCamera
✓ Offline Liveness Detection Blink, head-turn, challenge-response — passive + active anti-spoofing implemented	Face Detection: Ultra-Light (ONNX → TFLite)
✓ AWS Sync & Purge Mechanism Scope for S3 sync when connectivity restored; local purge after confirmed receipt	Face Recognition: MobileFaceNet INT8 Quantized
✓ Presentation & Technical Docs PPTX + PDF with model architecture, integration steps, and performance benchmarks	Liveness / PAD: MiniVGG + dlib + OpenCV
	Local Storage: SQLite (react-native-sqlite-storage)
	Encryption: AES-256 (react-native-encrypted-storage)
	Cloud Sync: AWS S3 via aws-sdk-js-v3
	Licenses: MIT / Apache-2.0 only — zero commercial

CONCLUSION

FaceGuard: The Complete Answer to Offline Biometric Authentication

- Fully offline — zero internet dependency at any step of authentication
- ~8 MB model footprint — well within Datalake 3.0's 20 MB budget
- Multi-layer anti-spoofing prevents photos, videos, and replay attacks
- AES-256 encrypted local storage — no raw biometrics ever persisted
- Automatic AWS S3 sync + purge when connectivity is restored
- 100% open-source (MIT / Apache-2.0) — no commercial licenses required